```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <unistd.h>
#include <errno.h>

// ANSI color codes for output
#define COLOR_RED           "\033[4;38;5;52m"
#define COLOR_GREEN         "\033[4;38;5;22m"
#define COLOR_YELLOW        "\033[4;38;5;58m"
#define COLOR_BLUE          "\033[4;38;5;17m"
#define COLOR_MAGENTA       "\033[4;38;5;53m"
#define COLOR_CYAN          "\033[4;38;5;23m"
#define COLOR_ORANGE        "\033[4;38;5;130m"
#define COLOR_BRIGHT_RED     "\033[0;91m"
#define COLOR_BRIGHT_GREEN   "\033[0;92m"
#define COLOR_BRIGHT_YELLOW  "\033[0;93m"
#define COLOR_BRIGHT_BLUE    "\033[0;94m"
#define COLOR_BRIGHT_MAGENTA "\033[0;95m"
#define COLOR_BRIGHT_CYAN    "\033[0;96m"
#define COLOR_BRIGHT_ORANGE         "\033[38;5;208m"

// Enum for recipe types
typedef enum {
    COOKIES,
    PANCAKES,
    PIZZA_DOUGH,
    SOFT_PRETZELS,
    CINNAMON_ROLLS
} RecipeType;

// Pantry structure
typedef struct {
    int flour;
    int sugar;
    int yeast;
    int baking_soda;
    int salt;
    int cinnamon;
} Pantry;

// Fridge structure
typedef struct {
    int eggs;
    int milk;
    int butter;
} Fridge;

// Kitchen structure
typedef struct {
    int bowls;
    int spoons;
    int mixers;
} Kitchen;

// RecipeCounter structure for assigning recipes
typedef struct {
    int next_recipe; // Counter to cycle through recipes
} RecipeCounter;

// BakerResources structure
typedef struct {
    int flour;
    int sugar;
    int yeast;
    int cinnamon;
    int baking_soda;
    int salt;
    int eggs;
    int milk;
    int butter;
    int has_bowl;
    int has_spoon;
    int has_mixer;
} BakerResources;

// BakerData structure
typedef struct {
    long baker_id;              // Unique ID for the baker
    RecipeType recipe_type;     // Assigned recipe
    Pantry* pantry_ptr;         // Pointer to shared pantry
    Fridge* fridge_ptr;         // Pointer to shared fridge
    Kitchen* kitchen_ptr;       // Pointer to shared kitchen
```

```
    RecipeCounter* counter_ptr; // Pointer to shared recipe counter
    int pantry_semid;           // Semaphore ID for pantry access
    int fridge_semid;           // Semaphore ID for fridge access
    int kitchen_semid;          // Semaphore ID for kitchen resources
    int recipe_semid;           // Semaphore ID for recipe counter
    BakerResources* resources;  // Pointer to baker's resources
    const char* color;
    int numRecipesCompleted;
    int has_been_ramsied;
} BakerData;

// Array of ANSI color codes
const char* COLORS[] = {
    COLOR_RED,
    COLOR_GREEN,
    COLOR_YELLOW,
    COLOR_BLUE,
    COLOR_MAGENTA,
    COLOR_CYAN,
    COLOR_ORANGE,
    COLOR_BRIGHT_RED,
    COLOR_BRIGHT_GREEN,
    COLOR_BRIGHT_YELLOW,
    COLOR_BRIGHT_BLUE,
    COLOR_BRIGHT_MAGENTA,
    COLOR_BRIGHT_CYAN,
    COLOR_BRIGHT_ORANGE
};
const int NUM_COLORS = 14;
const char* RESET_COLOR = "\033[0m";

// Recipe structures defining pantry and fridge item counts
typedef struct {
    int num_of_items_in_pantry;
    int num_of_items_in_fridge;
} cookies;

typedef struct {
    int num_of_items_in_pantry;
    int num_of_items_in_fridge;
} pancakes;

typedef struct {
    int num_of_items_in_pantry;
    int num_of_items_in_fridge;
} homemadePizzaDough;

typedef struct {
    int num_of_items_in_pantry;
    int num_of_items_in_fridge;
} softPretzels;

typedef struct {
    int num_of_items_in_pantry;
    int num_of_items_in_fridge;
} cinnamonRolls;

typedef struct {
    int semNumForBowls;
    int semNumForSpoons;
    int semNumForMixers;
} semNumValuesForKitchenResources;

semNumValuesForKitchenResources semaphoreNumValuesForKitchenResourc = {0, 1, 2};

// Initialize recipe requirements
cookies cookiesRecipe = {2, 2};                   // Flour, sugar, eggs, butter
pancakes pancakesRecipe = {4, 3};                 // Flour, sugar, baking soda, salt, eggs, milk, butter
homemadePizzaDough pizzaDoughRecipe = {3, 0};     // Flour, yeast, salt
softPretzels softPretzelsRecipe = {5, 1};         // Flour, sugar, yeast, salt, baking soda, butter
cinnamonRolls cinnamonRollsRecipe = {4, 2};       // Flour, sugar, yeast, cinnamon, eggs, butter

// Array of recipe names for printing
const char* recipe_names[] = {
    "Cookies",
    "Pancakes",
    "Pizza Dough",
    "Soft Pretzels",
    "Cinnamon Rolls"
};

// Decrement semaphore to acquire resource
void sem_wait(int semid, int sem_num) {
    struct sembuf op = {sem_num, -1, 0}; // Decrease semaphore by 1
    if (semop(semid, &op, 1) < 0) {
        perror("sem_wait failed");
    }
```

```
}

// Increment semaphore to release resource
void sem_signal(int semid, int sem_num) {
    struct sembuf op = {sem_num, 1, 0}; // Increase semaphore by 1
    if (semop(semid, &op, 1) < 0) {
        perror("sem_signal failed");
    }
}

// Print baker's current state with colored output
void print_baker_state(long baker_id, BakerResources* resources, RecipeType recipe_type) {
    const char* color = COLORS[baker_id % NUM_COLORS]; // Select color based on baker ID
    printf("%sBaker %ld (Recipe: %s) state: Flour=%d, Sugar=%d, Yeast=%d, BakingSoda=%d, "
           "Salt=%d, Cinnamon=%d, Eggs=%d, Milk=%d, Butter=%d, "
           "Bowl=%d, Spoon=%d, Mixer=%d%s\n",
           color, baker_id, recipe_names[recipe_type],
           resources->flour, resources->sugar, resources->yeast,
           resources->baking_soda, resources->salt, resources->cinnamon,
           resources->eggs, resources->milk, resources->butter,
           resources->has_bowl, resources->has_spoon, resources->has_mixer, RESET_COLOR);
}

// Thread function for each baker
void* baker_function(void* arg) {
    BakerData* data = (BakerData*)arg;
    long baker_id = data->baker_id;
    Pantry* pantry_ptr = data->pantry_ptr;
    Fridge* fridge_ptr = data->fridge_ptr;
    Kitchen* kitchen_ptr = data->kitchen_ptr;
    RecipeCounter* counter_ptr = data->counter_ptr;
    int pantry_semid = data->pantry_semid;
    int fridge_semid = data->fridge_semid;
    int kitchen_semid = data->kitchen_semid;
    int recipe_semid = data->recipe_semid;
    BakerResources* resources = data->resources;

    // Select recipe using shared counter
    sem_wait(recipe_semid, 0);
    // type cast that converts this integer into a value of the RecipeType enum,
    RecipeType recipe_type = (RecipeType)(counter_ptr->next_recipe % 5);
    counter_ptr->next_recipe++; // Increment for next baker
    sem_signal(recipe_semid, 0); // Unlock recipe counter

    data->recipe_type = recipe_type; // Store for printing state
    printf("Baker %ld assigned recipe: %s\n", baker_id, recipe_names[recipe_type]);
    while (data->numRecipesCompleted != 5){

        // Collect ingredients based on assigned recipe
        switch (recipe_type) {
            case COOKIES:
                // Cookies: 1 flour, 1 sugar, 1 egg, 1 butter
                sem_wait(pantry_semid, 0); // Enter pantry
                if (pantry_ptr->flour > 0) {
                    pantry_ptr->flour--;
                    resources->flour++;
                    printf("%sBaker %ld took 1 flour from pantry.%s\n", data->color,
                        data->baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no flour in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0); // Exit pantry
                print_baker_state(baker_id, resources, recipe_type);

                sem_wait(pantry_semid, 0);
                if (pantry_ptr->sugar > 0) {
                    pantry_ptr->sugar--;
                    resources->sugar++;
                    printf("%sBaker %ld took 1 sugar from pantry.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no sugar in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0);
                print_baker_state(baker_id, resources, recipe_type);

                sem_wait(fridge_semid, 0); // Enter fridge
                resources->milk++;
                printf("%sBaker %ld took 1 milk from fridge.%s\n", data->color,
                        baker_id, RESET_COLOR);
                sem_signal(fridge_semid, 0); // Exit fridge
                print_baker_state(baker_id, resources, recipe_type);

                sem_wait(fridge_semid, 0);
                resources->butter++;
                printf("%sBaker %ld took 1 butter from fridge.%s\n", data->color,
                        baker_id, RESET_COLOR);
```

```
                sem_signal(fridge_semid, 0);
                print_baker_state(baker_id, resources, recipe_type);
                printf("%sBaker %ld now has all the ingredients, time to get a spoon, bowl, and a mixer%s\n", data->color, baker_id,
RESET_COLOR);
                break;

            case PANCAKES:
            // Flour, sugar, baking soda, salt, eggs, milk, Butter
                sem_wait(pantry_semid, 0); // Enter pantry
                if (pantry_ptr->flour > 0) {
                    // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our
pantry.
                    resources->flour++;
                    printf("%sBaker %ld took 1 flour from pantry.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    // these edge cases for fridge and pantry items should never actually be executed (see line 259).
                    printf("Baker %ld found no flour in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0); // Exit pantry
                // we have acquired flour, lets get sugar
                print_baker_state(baker_id, resources, recipe_type);

                sem_wait(pantry_semid, 0); // Enter pantry
                if (pantry_ptr->flour > 0) {
                    // time to get sugar.
                    // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our
pantry.
                    resources->sugar++;
                    printf("%sBaker %ld took sugar from pantry.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no sugar in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0); // Exit pantry
                print_baker_state(baker_id, resources, recipe_type);
                // we now have flour and sugar, let's acquire baking soda

                sem_wait(pantry_semid, 0); // Enter pantry
                if (pantry_ptr->baking_soda > 0) {
                    // time to get sugar.
                    // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our
pantry.
                    resources->baking_soda++;
                    printf("%sBaker %ld took baking soda from pantry.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no baking soda in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0); // Exit pantry
                print_baker_state(baker_id, resources, recipe_type);
                // we now have flour, sugar and baking soda, let's acquire salt

                sem_wait(pantry_semid, 0); // Enter pantry
                if (pantry_ptr->salt > 0) {
                    // time to get sugar.
                    // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our
pantry.
                    resources->salt++;
                    printf("%sBaker %ld took salt from pantry.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no salt in pantry.\n", baker_id);
                }
                sem_signal(pantry_semid, 0); // Exit pantry
                print_baker_state(baker_id, resources, recipe_type);
                // we now have flour, sugar, baking soda, and salt. let's acquire an egg.

                sem_wait(fridge_semid, 0); // Enter fridge
                if (fridge_ptr->eggs > 0) {
                    resources->eggs++;
                    printf("%sBaker %ld took 1 egg from fridge. %s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no eggs in fridge.\n", baker_id);
                }
                sem_signal(fridge_semid, 0); // Exit fridge
                print_baker_state(baker_id, resources, recipe_type);

                // we now have flour, sugar, baking soda, salt, and egg. Let's acquire milk.
                sem_wait(fridge_semid, 0); // Enter fridge
                if (fridge_ptr->milk > 0) {
                    resources->milk++;
                    printf("%sBaker %ld took 1 milk from fridge.%s\n", data->color,
                        baker_id, RESET_COLOR);
                } else {
                    printf("Baker %ld found no milk in fridge.\n", baker_id);
```

```
            }
            sem_signal(fridge_semid, 0); // Exit fridge
            print_baker_state(baker_id, resources, recipe_type);

            // we now have flour, sugar, baking soda, salt, egg, and milk. Let's acquire butter.
            sem_wait(fridge_semid, 0); // Enter fridge
            if (fridge_ptr->butter > 0) {
                resources->butter++;
                printf("%sBaker %ld took 1 butter from fridge.%s\n", data->color,
                    baker_id, RESET_COLOR);
            } else {
                printf("Baker %ld found no butter in fridge.\n", baker_id);
            }
            sem_signal(fridge_semid, 0); // Exit fridge
            print_baker_state(baker_id, resources, recipe_type);
            printf("%sBaker %ld now has all the ingredients, time to get a spoon, bowl, and a mixer%s\n", data->color, baker_id,
RESET_COLOR);
            // We now have all 7 ingredients to make pancakes. Lets break out of the case and get the stuff to mix them together.
            break;

        case PIZZA_DOUGH:
            // Yeast, sugar, salt
            sem_wait(pantry_semid, 0); // Enter pantry
            // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our pantry.
            resources->yeast++;
            printf("%sBaker %ld took 1 yeast from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired yeast, lets get sugar

            sem_wait(pantry_semid, 0); // Enter pantry
            // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our pantry.
            resources->sugar++;
            printf("%sBaker %ld took 1 sugar from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired yeast, sugar, lets get salt

            sem_wait(pantry_semid, 0); // Enter pantry
            // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our pantry.
            resources->salt++;
            printf("%sBaker %ld took 1 salt from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // We now have all 3 ingredients to make pizza dough. Lets break out of the case and get the stuff to mix them together.
            printf("%sBaker %ld now has all the ingredients, time to get a spoon, bowl, and a mixer%s\n", data->color, baker_id,
RESET_COLOR);
            break;

        case SOFT_PRETZELS:
            // flour, sugar, salt, yeast, Baking soda, egg
            sem_wait(pantry_semid, 0); // Enter pantry
            // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our pantry.
            resources->flour++;
            printf("%sBaker %ld took 1 flour from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, lets get sugar

            sem_wait(pantry_semid, 0); // Enter pantry
            // Professor said assume we have unlimited pantry and fridge items, so we don't need to actually decrement it from our pantry.
            resources->sugar++;
            printf("%sBaker %ld took 1 sugar from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, lets get salt.

            sem_wait(pantry_semid, 0); // Enter pantry
            resources->salt++;
            printf("%sBaker %ld took 1 salt from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, and salt. lets get yeast.

            sem_wait(pantry_semid, 0); // Enter pantry
            resources->yeast++;
            printf("%sBaker %ld took 1 yeast from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, salt, and yeast. lets get baking soda.
```

```
                sem_wait(pantry_semid, 0); // Enter pantry
                resources->baking_soda++;
                printf("%sBaker %ld took 1 baking soda from pantry.%s\n", data->color,
                baker_id, RESET_COLOR);
                sem_signal(pantry_semid, 0); // Exit pantry
                print_baker_state(baker_id, resources, recipe_type);
                // we have acquired flour, sugar, salt, yeast, and baking soda. lets get an egg.

                sem_wait(fridge_semid, 0); // Enter fridge
                resources->eggs++;
                printf("%sBaker %ld took 1 egg from fridge.%s\n", data->color,
                        baker_id, RESET_COLOR);
                sem_signal(fridge_semid, 0); // Exit fridge
                print_baker_state(baker_id, resources, recipe_type);
                // we have acquired flour, sugar, salt, yeast, baking soda, and an egg. Lets break out of the case and get the stuff to mix
them together.
                printf("%sBaker %ld now has all the ingredients, time to get a spoon, bowl, and a mixer%s\n", data->color, baker_id,
RESET_COLOR);
                break;


            case CINNAMON_ROLLS:
            // flour, sugar, salt, cinnamon, eggs, butter
            sem_wait(pantry_semid, 0); // Enter pantry
            resources->flour++;
            printf("%sBaker %ld took 1 flour from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour. lets get an sugar.

            sem_wait(pantry_semid, 0); // Enter pantry
            resources->sugar++;
            printf("%sBaker %ld took 1 sugar from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar. lets get an salt.

            sem_wait(pantry_semid, 0); // Enter pantry
            resources->salt++;
            printf("%sBaker %ld took 1 salt from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, salt. lets get a cinnamon.

            sem_wait(pantry_semid, 0); // Enter pantry
            resources->cinnamon++;
            printf("%sBaker %ld took 1 cinnamon from pantry.%s\n", data->color,
            baker_id, RESET_COLOR);
            sem_signal(pantry_semid, 0); // Exit pantry
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, salt, cinnamon. lets get an egg.

            sem_wait(fridge_semid, 0); // Enter fridge
            resources->eggs++;
            printf("%sBaker %ld took 1 egg from fridge.%s\n", data->color,
                    baker_id, RESET_COLOR);
            sem_signal(fridge_semid, 0); // Exit fridge
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, salt, cinnamon, and eggs. lets get an butter.

            sem_wait(fridge_semid, 0); // Enter fridge
            resources->butter++;
            printf("%sBaker %ld took 1 butter from fridge.%s\n", data->color,
                    baker_id, RESET_COLOR);
            sem_signal(fridge_semid, 0); // Exit fridge
            print_baker_state(baker_id, resources, recipe_type);
            // we have acquired flour, sugar, salt, cinnamon, eggs, and butter. Lets break out of the case and get the stuff to mix them
together.
            printf("%sBaker %ld now has all the ingredients, time to get a spoon, bowl, and a mixer%s\n", data->color, baker_id, RESET_COLOR);
            break;
        }

        // Collect kitchen resources
        // All recipes need a bowl, mixer, and spoon
        printf("%sBaker %ld is trying to get a mixer%s\n", data->color, baker_id, RESET_COLOR);
        sem_wait(kitchen_semid, 2); // Acquire mixer
            resources->has_mixer = 1;
            printf("%sBaker %ld took a mixer from kitchen. Mixers left: %d%s\n", data->color,
                baker_id, kitchen_ptr->mixers, RESET_COLOR);
        print_baker_state(baker_id, resources, recipe_type);


        printf("%sBaker %ld is trying to get a bowl%s\n", data->color, baker_id, RESET_COLOR);
```

```
        sem_wait(kitchen_semid, 0); // Acquire bowl
            resources->has_bowl = 1;
            printf("%sBaker %ld took a bowl from kitchen. Bowls left: %d%s\n", data->color,
                baker_id, kitchen_ptr->bowls, RESET_COLOR);
        print_baker_state(baker_id, resources, recipe_type);

        printf("%sBaker %ld is trying to get a spoon%s\n", data->color, baker_id, RESET_COLOR);
        sem_wait(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForSpoons); // Acquire spoon
            resources->has_spoon = 1;
            printf("%sBaker %ld took a spoon from kitchen. Spoons left: %d%s\n", data->color,
                baker_id, kitchen_ptr->spoons, RESET_COLOR);
        print_baker_state(baker_id, resources, recipe_type);

        // Let's make baker 0 the ramsied chance one.
        if (baker_id == 0){
            if (recipe_type == COOKIES && data->has_been_ramsied == 0){
                printf("GET RAMSIED!! RESTARTTTTTT!!!\n");
                printf("%sBaker %ld got RAMSIED and has to release all resources and restart their current recipe, %s, from scratch%s\n", data-
>color, baker_id, recipe_names[recipe_type], RESET_COLOR);
                data->resources->baking_soda = 0;
                data->resources->butter = 0;
                data->resources->cinnamon = 0;
                data->resources->eggs = 0;
                data->resources->flour = 0;

                // Return mixer resource to allow other bakers to use them
                if (resources->has_mixer) {
                    sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForMixers); // Release mixer
                    data->resources->has_mixer = 0;
                }

                // Return spoon resource to allow other bakers to use them
                if (resources->has_spoon) {
                    sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForSpoons); // Release spoon
                    data->resources->has_spoon = 0;
                }

                // Return bowl resource to allow other bakers to use them
                if (resources->has_bowl) {
                    sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForBowls); // Release bowl
                    data->resources->has_bowl = 0;
                }

                data->resources->milk = 0;
                data->resources->salt = 0;
                data->resources->sugar = 0;
                data->resources->yeast = 0;
                data->has_been_ramsied = 1;
                continue;
            }
        }

        printf("%sBaker %ld is mixing the ingredients all together%s\n", data->color, baker_id, RESET_COLOR);
        printf("%sBaker %ld has finished mixing the ingredients together%s\n", data->color, baker_id, RESET_COLOR);

        // Return mixer resource to allow other bakers to use them
        if (resources->has_mixer) {
            sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForMixers); // Release mixer
            data->resources->has_mixer = 0;
        }

        // Return spoon resource to allow other bakers to use them
        if (resources->has_spoon) {
            sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForSpoons); // Release spoon
            data->resources->has_spoon = 0;
        }

        // Return bowl resource to allow other bakers to use them
        if (resources->has_bowl) {
            sem_signal(kitchen_semid, semaphoreNumValuesForKitchenResourc.semNumForBowls); // Release bowl
            data->resources->has_bowl = 0;
        }

        printf("%sBaker %ld is now going to use the oven%s\n", data->color, baker_id, RESET_COLOR);
        sleep(2);
        printf("%sBaker %lds Recipe %s has finished cooking in the oven.%s\n", data->color, data->baker_id,recipe_names[recipe_type],
RESET_COLOR);
        //recipe completion
        printf("\n\n%sBaker %ld completed %s.%s\n", data->color, baker_id, recipe_names[recipe_type], RESET_COLOR);
        data->numRecipesCompleted++;
        printf("%sBaker %ld has completed %d/5 recipes.%s\n\n", data->color, baker_id, data->numRecipesCompleted, RESET_COLOR);

        // we finished a recipe lets increment the number of recipes completed.
        recipe_type++;
        recipe_type = recipe_type % 5;
        data->recipe_type = recipe_type; // Store for printing state
        printf("%sBaker %ld assigned recipe: %s%s\n", data->color, baker_id, recipe_names[recipe_type], RESET_COLOR);
```

```
        // release all their current stuff they're holding.
        data->resources->baking_soda = 0;
        data->resources->butter = 0;
        data->resources->cinnamon = 0;
        data->resources->eggs = 0;
        data->resources->flour = 0;
        data->resources->milk = 0;
        data->resources->salt = 0;
        data->resources->sugar = 0;
        data->resources->yeast = 0;

    }


    // Free thread-specific memory
    free(data->resources);
    free(data);
    return NULL;
}

// Main function to set up shared memory, semaphores, and threads
int main() {
    int num_bakers;

    // Prompt user for number of bakers
    printf("Enter the number of bakers to create: 1 - 14\n");
    if (scanf("%d", &num_bakers) != 1 || num_bakers <= 0) {
        printf("Please enter a positive number.\n");
        return 1;
    }

    // Allocate memory for baker threads
    pthread_t* bakers = malloc(num_bakers * sizeof(pthread_t));
    if (bakers == NULL) {
        perror("Failed to allocate memory for bakers");
        return 1;
    }

    // Create semaphore for pantry
    int pantry_semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
    if (pantry_semid < 0) {
        perror("Pantry semget failed");
        free(bakers);
        return 1;
    }
    if (semctl(pantry_semid, 0, SETVAL, 1) < 0) {
        perror("Pantry semctl SETVAL failed");
        free(bakers);
        return 1;
    }

    // Create semaphore for fridge
    int fridge_semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
    if (fridge_semid < 0) {
        perror("Fridge semget failed");
        semctl(pantry_semid, 0, IPC_RMID);
        free(bakers);
        return 1;
    }
    if (semctl(fridge_semid, 0, SETVAL, 2) < 0) {
        perror("Fridge semctl SETVAL failed");
        semctl(pantry_semid, 0, IPC_RMID);
        semctl(fridge_semid, 0, IPC_RMID);
        free(bakers);
        return 1;
    }

    // Create semaphore for kitchen
    int kitchen_semid = semget(IPC_PRIVATE, 3, IPC_CREAT | 0666);
    if (kitchen_semid < 0) {
        perror("Kitchen semget failed");
        semctl(pantry_semid, 0, IPC_RMID);
        semctl(fridge_semid, 0, IPC_RMID);
        free(bakers);
        return 1;
    }
    if (semctl(kitchen_semid, 0, SETVAL, 3) < 0) { // 3 bowls
        perror("Kitchen semctl SETVAL bowls failed");
        semctl(pantry_semid, 0, IPC_RMID);
        semctl(fridge_semid, 0, IPC_RMID);
        semctl(kitchen_semid, 0, IPC_RMID);
        free(bakers);
        return 1;
    }
    if (semctl(kitchen_semid, 1, SETVAL, 5) < 0) { // 5 spoons
        perror("Kitchen semctl SETVAL spoons failed");
        semctl(pantry_semid, 0, IPC_RMID);
```

```
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    free(bakers);
    return 1;
}
if (semctl(kitchen_semid, 2, SETVAL, 2) < 0) { // 2 mixers
    perror("Kitchen semctl SETVAL mixers failed");
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    free(bakers);
    return 1;
}

// Create semaphore for recipe counter
int recipe_semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
if (recipe_semid < 0) {
    perror("Recipe semget failed");
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    free(bakers);
    return 1;
}
if (semctl(recipe_semid, 0, SETVAL, 1) < 0) {
    perror("Recipe semctl SETVAL failed");
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    free(bakers);
    return 1;
}

// Create shared memory for pantry
int pantry_shmid = shmget(IPC_PRIVATE, sizeof(Pantry), IPC_CREAT | 0666);
if (pantry_shmid < 0) {
    perror("Pantry shmget failed");
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    free(bakers);
    return 1;
}
Pantry* pantry_ptr = (Pantry*)shmat(pantry_shmid, NULL, 0);
if (pantry_ptr == (void*)-1) {
    perror("Pantry shmat failed");
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
// Initialize pantry with 50
//  This was from a previous idea b4 we knew that the pantry and fridge had an unlimited
// amount of items. We left it because of some if statements from the multi-threaded function
// and we wanted to spend our time finishing the project, rather than debugging it.
pantry_ptr->flour = 50;
pantry_ptr->sugar = 50;
pantry_ptr->yeast = 50;
pantry_ptr->baking_soda = 50;
pantry_ptr->salt = 50;
pantry_ptr->cinnamon = 50;

// Create shared memory for fridge
int fridge_shmid = shmget(IPC_PRIVATE, sizeof(Fridge), IPC_CREAT | 0666);
if (fridge_shmid < 0) {
    perror("Fridge shmget failed");
    shmdt(pantry_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
Fridge* fridge_ptr = (Fridge*)shmat(fridge_shmid, NULL, 0);
if (fridge_ptr == (void*)-1) {
    perror("Fridge shmat failed");
    shmdt(pantry_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
```

```
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    shmctl(fridge_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
// Initialize fridge with 50 units of each ingredient
//  This was from a previous idea b4 we knew that the pantry and fridge had an unlimited
// amount of items. We left it because of some if statements from the multi-threaded function
// and we wanted to spend our time finishing the project, rather than debugging it.

fridge_ptr->eggs = 50;
fridge_ptr->milk = 50;
fridge_ptr->butter = 50;

// Create shared memory for kitchen
int kitchen_shmid = shmget(IPC_PRIVATE, sizeof(Kitchen), IPC_CREAT | 0666);
if (kitchen_shmid < 0) {
    perror("Kitchen shmget failed");
    shmdt(pantry_ptr);
    shmdt(fridge_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    shmctl(fridge_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
Kitchen* kitchen_ptr = (Kitchen*)shmat(kitchen_shmid, NULL, 0);
if (kitchen_ptr == (void*)-1) {
    perror("Kitchen shmat failed");
    shmdt(pantry_ptr);
    shmdt(fridge_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    shmctl(fridge_shmid, IPC_RMID, NULL);
    shmctl(kitchen_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
// Initialize kitchen resources
kitchen_ptr->bowls = 3;
kitchen_ptr->spoons = 5;
kitchen_ptr->mixers = 2;

// Create shared memory for recipe counter
int counter_shmid = shmget(IPC_PRIVATE, sizeof(RecipeCounter), IPC_CREAT | 0666);
if (counter_shmid < 0) {
    perror("Counter shmget failed");
    shmdt(pantry_ptr);
    shmdt(fridge_ptr);
    shmdt(kitchen_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    shmctl(fridge_shmid, IPC_RMID, NULL);
    shmctl(kitchen_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
RecipeCounter* counter_ptr = (RecipeCounter*)shmat(counter_shmid, NULL, 0);
if (counter_ptr == (void*)-1) {
    perror("Counter shmat failed");
    shmdt(pantry_ptr);
    shmdt(fridge_ptr);
    shmdt(kitchen_ptr);
    semctl(pantry_semid, 0, IPC_RMID);
    semctl(fridge_semid, 0, IPC_RMID);
    semctl(kitchen_semid, 0, IPC_RMID);
    semctl(recipe_semid, 0, IPC_RMID);
    shmctl(pantry_shmid, IPC_RMID, NULL);
    shmctl(fridge_shmid, IPC_RMID, NULL);
    shmctl(kitchen_shmid, IPC_RMID, NULL);
    shmctl(counter_shmid, IPC_RMID, NULL);
    free(bakers);
    return 1;
}
// Initialize recipe counter
// Seed the random number generator with current time
srand(time(NULL));
```

```
// Get a random number from 0 to 5
int randomNumber = rand() % 6;

counter_ptr->next_recipe = randomNumber;
// counter_ptr->next_recipe = 0; // REPLACE THIS LINE WITH THE COMMENTED OUT LINE ABOVE WHEN READY FOR RECIPES TO BE RANDOMLY PICKED.


// Create baker threads
for (long i = 0; i < num_bakers; i++) {
    // Allocate memory for thread data
    BakerData* bdata = malloc(sizeof(BakerData));
    if (bdata == NULL) {
        perror("Failed to allocate BakerData");
        shmdt(pantry_ptr);
        shmdt(fridge_ptr);
        shmdt(kitchen_ptr);
        shmdt(counter_ptr);
        semctl(pantry_semid, 0, IPC_RMID);
        semctl(fridge_semid, 0, IPC_RMID);
        semctl(kitchen_semid, 0, IPC_RMID);
        semctl(recipe_semid, 0, IPC_RMID);
        shmctl(pantry_shmid, IPC_RMID, NULL);
        shmctl(fridge_shmid, IPC_RMID, NULL);
        shmctl(kitchen_shmid, IPC_RMID, NULL);
        shmctl(counter_shmid, IPC_RMID, NULL);
        free(bakers);
        return 1;
    }
    // Initialize thread data
    bdata->baker_id = i;
    bdata->recipe_type = 0; // Will be set in baker_function
    bdata->pantry_ptr = pantry_ptr;
    bdata->fridge_ptr = fridge_ptr;
    bdata->kitchen_ptr = kitchen_ptr;
    bdata->counter_ptr = counter_ptr;
    bdata->pantry_semid = pantry_semid;
    bdata->fridge_semid = fridge_semid;
    bdata->kitchen_semid = kitchen_semid;
    bdata->recipe_semid = recipe_semid;
    const char* colorForThisBaker = COLORS[bdata->baker_id % NUM_COLORS]; // Select color based on baker ID
    bdata->color = colorForThisBaker;
    bdata->numRecipesCompleted = 0;
    bdata->has_been_ramsied = 0;



    bdata->resources = malloc(sizeof(BakerResources));
    if (bdata->resources == NULL) {
        perror("Failed to allocate BakerResources");
        free(bdata);
        shmdt(pantry_ptr);
        shmdt(fridge_ptr);
        shmdt(kitchen_ptr);
        shmdt(counter_ptr);
        semctl(pantry_semid, 0, IPC_RMID);
        semctl(fridge_semid, 0, IPC_RMID);
        semctl(kitchen_semid, 0, IPC_RMID);
        semctl(recipe_semid, 0, IPC_RMID);
        shmctl(pantry_shmid, IPC_RMID, NULL);
        shmctl(fridge_shmid, IPC_RMID, NULL);
        shmctl(kitchen_shmid, IPC_RMID, NULL);
        shmctl(counter_shmid, IPC_RMID, NULL);
        free(bakers);
        return 1;
    }
    // Initialize baker's resources to zero
    bdata->resources->flour = 0;
    bdata->resources->sugar = 0;
    bdata->resources->yeast = 0;
    bdata->resources->cinnamon = 0;
    bdata->resources->baking_soda = 0;
    bdata->resources->salt = 0;
    bdata->resources->eggs = 0;
    bdata->resources->milk = 0;
    bdata->resources->butter = 0;
    bdata->resources->has_bowl = 0;
    bdata->resources->has_spoon = 0;
    bdata->resources->has_mixer = 0;

    // Create baker thread
    if (pthread_create(&bakers[i], NULL, baker_function, (void*)bdata) != 0) {
        perror("Baker creation failed");
        free(bdata->resources);
        free(bdata);
        shmdt(pantry_ptr);
        shmdt(fridge_ptr);
```

```
            shmdt(kitchen_ptr);
            shmdt(counter_ptr);
            semctl(pantry_semid, 0, IPC_RMID);
            semctl(fridge_semid, 0, IPC_RMID);
            semctl(kitchen_semid, 0, IPC_RMID);
            semctl(recipe_semid, 0, IPC_RMID);
            shmctl(pantry_shmid, IPC_RMID, NULL);
            shmctl(fridge_shmid, IPC_RMID, NULL);
            shmctl(kitchen_shmid, IPC_RMID, NULL);
            shmctl(counter_shmid, IPC_RMID, NULL);
            free(bakers);
            return 1;
        }
    }

    // Wait for all baker threads to complete
    for (int i = 0; i < num_bakers; i++) {
        pthread_join(bakers[i], NULL);
    }

    // Print final state of pantry, fridge, and kitchen
    printf("\nFinal state:\n");
    printf("Pantry: Flour=%d, Sugar=%d, Yeast=%d, Baking Soda=%d, Salt=%d, Cinnamon=%d\n",
           pantry_ptr->flour, pantry_ptr->sugar, pantry_ptr->yeast,
           pantry_ptr->baking_soda, pantry_ptr->salt, pantry_ptr->cinnamon);
    printf("Fridge: Eggs=%d, Milk=%d, Butter=%d\n",
           fridge_ptr->eggs, fridge_ptr->milk, fridge_ptr->butter);
    printf("Kitchen: Bowls=%d, Spoons=%d, Mixers=%d\n",
           kitchen_ptr->bowls, kitchen_ptr->spoons, kitchen_ptr->mixers);

    // Clean up resources
    semctl(pantry_semid, 0, IPC_RMID);   // Remove pantry semaphore
    semctl(fridge_semid, 0, IPC_RMID);   // Remove fridge semaphore
    semctl(kitchen_semid, 0, IPC_RMID);  // Remove kitchen semaphore
    semctl(recipe_semid, 0, IPC_RMID);   // Remove recipe semaphore
    shmdt(pantry_ptr);                   // Detach pantry shared memory
    shmdt(fridge_ptr);                   // Detach fridge shared memory
    shmdt(kitchen_ptr);                  // Detach kitchen shared memory
    shmdt(counter_ptr);                  // Detach counter shared memory
    shmctl(pantry_shmid, IPC_RMID, NULL);  // Remove pantry shared memory
    shmctl(fridge_shmid, IPC_RMID, NULL);  // Remove fridge shared memory
    shmctl(kitchen_shmid, IPC_RMID, NULL); // Remove kitchen shared memory
    shmctl(counter_shmid, IPC_RMID, NULL); // Remove counter shared memory
    free(bakers);                        // Free thread array
    printf("Cleanup complete.\n");

    return 0;
}
```